

PointerPro Animate

Generated by Doxygen 1.9.8

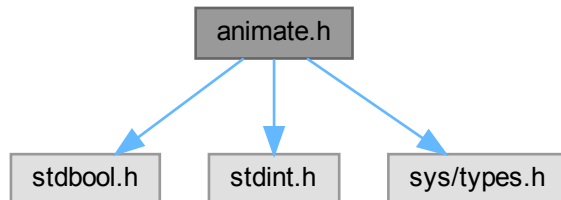
Chapter 1

File Documentation

1.1 animate.h File Reference

```
#include <stdbool.h>
#include <stdint.h>
#include <sys/types.h>
```

Include dependency graph for animate.h:



Typedefs

- typedef uint32_t [color_t](#)
Representation of a pixel.
- typedef void(* [animate_fn](#)) (void *priv, ssize_t *x, ssize_t *y, float t)
Custom animation function for optional extension.

Functions

- struct canvas * [animate_create_canvas](#) (size_t height, size_t width, [color_t](#) background_color)
Create the canvas on which our animation will be built.
- void [animate_destroy_canvas](#) (struct canvas *canvas)
Clean up a canvas.
- struct sprite * [animate_create_sprite](#) (const char *file)

- Create a sprite from the provided input bitmap file.*
 - struct sprite * [animate_create_rectangle](#) (size_t width, size_t height, [color_t](#) c, bool filled)
- Create a sprite that represents a rectangle.*
 - struct sprite * [animate_create_circle](#) (size_t radius, [color_t](#) c, bool filled)
- Create a sprite that represents a circle.*
 - bool [animate_destroy_sprite](#) (struct sprite *sprite)
- Frees all memory associated with a sprite.*
 - struct sprite_placement * [animate_place_sprite](#) (struct canvas *canvas, struct sprite *sprite, ssize_t x, ssize_t y)
- Places a sprite on the top layer of the canvas.*
 - void [animate_placement_up](#) (struct sprite_placement *sprite_placement)
- Raise a sprite one level towards the top layer.*
 - void [animate_placement_down](#) (struct sprite_placement *sprite_placement)
- Lower a sprite one level towards the bottom layer.*
 - void [animate_placement_top](#) (struct sprite_placement *sprite_placement)
- Raise the sprite to the top-most layer.*
 - void [animate_placement_bottom](#) (struct sprite_placement *sprite_placement)
- Lower the sprite to the bottom-most layer.*
 - void [animate_destroy_placement](#) (struct sprite_placement *sprite_placement)
- Removes the placement from its associated canvas and free resources.*
 - void [animate_set_animation_params](#) (struct sprite_placement *sprite_placement, ssize_t vx, ssize_t vy, ssize_t ax, ssize_t ay)
- Configure sprite animation.*
 - void [animate_set_animation_funtion](#) (struct sprite_placement *sprite_placement, [animate_fn](#), void *priv)
- size_t [animate_frame_size_bytes](#) (struct canvas *canvas)
- Return the size in bytes of an animation frame.*
 - void [animate_generate_frame](#) (const struct canvas *canvas, size_t frame, size_t frame_rate, void *buf)
- Generate a frame of the animation.*

1.1.1 Typedef Documentation

1.1.1.1 animate_fn

```
typedef void(* animate_fn) (void *priv, ssize_t *x, ssize_t *y, float t)
```

Custom animation function for optional extension.

Parameters

<i>priv</i>	a handle to the private data provided when the function was registered
<i>x</i>	a reference to the starting position. On return, this should contain the target sprite position for the frame in question.
<i>y</i>	a reference to the starting position. On return, this should contain the position for the frame in question.
<i>t</i>	the current time since the start of the animation.

1.1.1.2 color_t

```
typedef uint32_t color_t
```

Representation of a pixel.

1.1.2 Function Documentation

1.1.2.1 animate_create_canvas()

```
struct canvas * animate_create_canvas (
    size_t height,
    size_t width,
    color_t background_color )
```

Create the canvas on which our animation will be built.

Parameters

<i>height</i>	height of the drawing area in pixels
<i>width</i>	width of the drawing area in pixels
<i>background_color</i>	the colour to use when no sprites overlap a pixel.

Returns

Abstract data type for the canvas. The memory associated this struct is allocated by this function and cleaned up by the corresponding destroy function

1.1.2.2 animate_create_circle()

```
struct sprite * animate_create_circle (
    size_t radius,
    color_t c,
    bool filled )
```

Create a sprite that represents a circle.

We currently only support filled circles. The circle pixels can be determined by the following formula: $x*x + y*y < r*r$ Where x is the x offset from the center of the circle y is the y offset from the center of the circle r is the radius of the circle

Parameters

<i>radius</i>	the radius of the circle
<i>c</i>	the colour of the circle
<i>filled</i>	reserved for future use

Returns

A handle to the created sprite

1.1.2.3 animate_create_rectangle()

```
struct sprite * animate_create_rectangle (
    size_t width,
```

```
size_t height,  
color_t c,  
bool filled )
```

Create a sprite that represents a rectangle.

Parameters

<i>width</i>	the width of the rectangle
<i>height</i>	the height of the rectangle
<i>c</i>	the colour of the rectangle
<i>filled</i>	true if the rectangle should be filled, otherwise false

Returns

A handle to the created sprite

1.1.2.4 `animate_create_sprite()`

```
struct sprite * animate_create_sprite (  
    const char * file )
```

Create a sprite from the provided input bitmap file.

The file format can be assumed to be ARGB32. The bitmap content may be loaded only on first access, however it must only be loaded once. NOTE: the order of image rows differs to what we expect. The first pixel in the bmp file format is on the bottom left while the last pixel is the top right. For simplicity, reverse the order of rows while loading the bitmap.

Parameters

<i>file</i>	the name of the bitmap file to be loaded
-------------	--

1.1.2.5 `animate_destroy_canvas()`

```
void animate_destroy_canvas (  
    struct canvas * canvas )
```

Clean up a canvas.

Frees all memory associated with the canvas, including placements, but not sprites

Parameters

<i>canvas</i>	the canvas to clean up
---------------	------------------------

1.1.2.6 animate_destroy_placement()

```
void animate_destroy_placement (
    struct sprite_placement * sprite_placement )
```

Removes the placement from its associated canvas and free resources.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.7 animate_destroy_sprite()

```
bool animate_destroy_sprite (
    struct sprite * sprite )
```

Frees all memory associated with a sprite.

Parameters

<i>sprite</i>	the sprite to clean up
---------------	------------------------

Returns

0 on success, otherwise an error occurred. The only error possible is that the sprite is still in use

1.1.2.8 animate_frame_size_bytes()

```
size_t animate_frame_size_bytes (
    struct canvas * canvas )
```

Return the size in bytes of an animation frame.

This is useful for the caller to determine how much memory to allocate when combining sprites into a frame.

Parameters

<i>canvas</i>	the canvas in question
---------------	------------------------

Returns

the size, in bytes, of a frame for this canvas

1.1.2.9 animate_generate_frame()

```
void animate_generate_frame (
    const struct canvas * canvas,
```

```

size_t frame,
size_t frame_rate,
void * buf )

```

Generate a frame of the animation.

Parameters

<i>canvas</i>	the canvas describing the animation
<i>frame</i>	the frame number (starting from 0)
<i>frame_rate</i>	the number of frames per second in the animation
<i>buf</i>	a buffer provided by the user that is large enough to hold a frame. This buffer will contain the frame data on return.

1.1.2.10 `animate_place_sprite()`

```

struct sprite_placement * animate_place_sprite (
    struct canvas * canvas,
    struct sprite * sprite,
    ssize_t x,
    ssize_t y )

```

Places a sprite on the top layer of the canvas.

Parameters

<i>canvas</i>	the canvas to place the sprite on
<i>sprite</i>	the sprite to place
<i>x</i>	the x coordinate of the canvas to which the top left pixel of the sprite should be placed
<i>y</i>	the y coordinate of the canvas to which the top left pixel of the sprite should be placed

Returns

A handle to a sprite placement.

1.1.2.11 `animate_placement_bottom()`

```

void animate_placement_bottom (
    struct sprite_placement * sprite_placement )

```

Lower the sprite to the bottom-most layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.12 `animate_placement_down()`

```
void animate_placement_down (
    struct sprite_placement * sprite_placement )
```

Lower a sprite one level towards the bottom layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.13 `animate_placement_top()`

```
void animate_placement_top (
    struct sprite_placement * sprite_placement )
```

Raise the sprite to the top-most layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.14 `animate_placement_up()`

```
void animate_placement_up (
    struct sprite_placement * sprite_placement )
```

Raise a sprite one level towards the top layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.15 `animate_set_animation_funtion()`

```
void animate_set_animation_funtion (
    struct sprite_placement * sprite_placement,
    animate_fn ,
    void * priv )
```

@breif [Optional extension] define animation using the provided function pointer

Parameters

<i>sprite_placement</i>	a handle to the sprite placement
<i>animate_fn</i>	a custom function to call when determining the position of the sprite in each frame.
<i>priv</i>	private data that is passed to the provided function on call.
<i>ax</i>	the acceleration in the x direction
<i>ay</i>	the acceleration in the y direction

1.1.2.16 animate_set_animation_params()

```
void animate_set_animation_params (
    struct sprite_placement * sprite_placement,
    ssize_t vx,
    ssize_t vy,
    ssize_t ax,
    ssize_t ay )
```

Configure sprite animation.

Currently, we support only physics simulation (velocity and acceleration)

Parameters

<i>sprite_placement</i>	a handle to the sprite placement
<i>vx</i>	the initial velocity in the x direction
<i>vy</i>	the initial velocity in the y direction
<i>ax</i>	the acceleration in the x direction
<i>ay</i>	the acceleration in the y direction

1.2 animate.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ANIMATE_H
00002 #define ANIMATE_H
00003
00004 #include <stdbool.h> /* for bool */
00005 #include <stdint.h> /* for uint32_t */
00006 #include <sys/types.h> /* for size_t/ssize_t */
00007
00015 struct sprite;
00016 struct canvas;
00017 struct sprite_placement;
00018
00022 typedef uint32_t color_t;
00023
00035 typedef void (*animate_fn)(void* priv, ssize_t* x, ssize_t* y, float t);
00036
00037 // Inline functions are often preferred over macros as types will be checked
00041 static inline color_t animate_color_rgb(unsigned r, unsigned g, unsigned b) {
00042     return ((r & 0xff) << 16) | ((g & 0xff) << 8) | ((b & 0xff) << 0);
00043 }
00044
00048 static inline color_t animate_color_argb(unsigned a,
00049     unsigned r, unsigned g, unsigned b) {
00050     return animate_color_rgb(r, g, b) | ((a & 0xff) << 24);
00051 }
00052
00063 struct canvas* animate_create_canvas(size_t height, size_t width,
00064     color_t background_color);
00065
00073 void animate_destroy_canvas(struct canvas* canvas);
00074
00086 struct sprite* animate_create_sprite(const char* file);
00087
00098 struct sprite* animate_create_rectangle(size_t width, size_t height, color_t c,
00099     bool filled);
00100
00117 struct sprite* animate_create_circle(size_t radius, color_t c, bool filled);
00118
00126 bool animate_destroy_sprite(struct sprite* sprite);
00127
00139 struct sprite_placement* animate_place_sprite(struct canvas* canvas,
00140     struct sprite* sprite,
00141     ssize_t x, ssize_t y);
00142
```

```
00148 void animate_placement_up(struct sprite_placement* sprite_placement);
00149
00155 void animate_placement_down(struct sprite_placement* sprite_placement);
00156
00162 void animate_placement_top(struct sprite_placement* sprite_placement);
00163
00169 void animate_placement_bottom(struct sprite_placement* sprite_placement);
00170
00176 void animate_destroy_placement(struct sprite_placement* sprite_placement);
00177
00178
00189 void animate_set_animation_params(struct sprite_placement* sprite_placement,
00190                                   ssize_t vx, ssize_t vy,
00191                                   ssize_t ax, ssize_t ay);
00192
00204 void animate_set_animation_funtion(struct sprite_placement* sprite_placement,
00205                                    animate_fn, void* priv);
00206
00215 size_t animate_frame_size_bytes(struct canvas* canvas);
00216
00227 void animate_generate_frame(const struct canvas* canvas,
00228                             size_t frame, size_t frame_rate,
00229                             void* buf);
00230
00231
00232 #endif /* ANIMATE_H */
```

